Phase 1 — Identity Architecture & Prerequisites

This document contains the complete Phase 1 deliverables for the Patient Portal Identity project: ERD, PostgreSQL schema (DDL + migrations), Microsoft Entra External ID mapping & app registration checklist, Auth Service API contract, security & compliance notes, epic-level tasks, and acceptance criteria. Use this as the canonical reference while implementing Phase 1.

---

1. Goals for Phase 1

- Define canonical identity model and relationships (UPI-first).
- Implement PostgreSQL schema and migrations for `patients`, `users`, `credentials`, `external_identities`, `proxy_access`, `empi_records`, `staff_invites`, `verification_evidence`, `audit_log`, `merge_tickets`, etc.
- Configure Microsoft Entra External ID tenant attributes and app registrations required for Phase 1 integration (OIDC, custom policies stub).
- Define .NET Core Auth Service API surface for entitlement mapping, token exchange, and EMPI stubs.
- Establish acceptance criteria, security controls, and infra prerequisites.

---

2. High-level ERD (textual)

```
[patients] 1---* [users] *---* [external_identities]
   |                     \         |
   |                      \        |
   |                       \       |
   *                        [credentials]
   |
   *---* [proxy_access]
   |
   *---* [verification_evidence]
   |
   *---* [empi_records]

[audit_log]
[merge_tickets]
[staff_invites]
```

Notes: - `patients` is the canonical clinical entity (UPI unique). - `users` are authentication principals—patients and caregivers are both users. `patient_id` may be NULL for caregiver or staff-only accounts. - `external_identities` map Entra/social provider subjects to `users`. - `credentials` holds optional

local credentials (passwords, pin_hash, webauthn references). - `proxy_access` maps patient -> delegate user with scopes and expiration.

---

3. PostgreSQL Schema (DDL) — Phase 1 core tables

Below DDL is intended for PostgreSQL 14+. Use `pgcrypto` extension for UUID generation and `pg_trgm` for name searching on EMPI.

```
-- Enable extensions
CREATE EXTENSION IF NOT EXISTS "pgcrypto";
CREATE EXTENSION IF NOT EXISTS pg_trgm;

-- patients
CREATE TABLE patients (
  patient_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  upi TEXT NOT NULL UNIQUE,
  first_name TEXT,
  middle_name TEXT,
  last_name TEXT,
  dob DATE,
  gender TEXT,
  addresses JSONB,
  identifiers JSONB, -- e.g., [{"system":"MRN","value":"12345"},
{"system":"SSN","value":"***"}]
  empi_score NUMERIC,
  empi_status TEXT DEFAULT 'unknown',
  status TEXT DEFAULT 'provisioned', --
'provisioned','active','merged','archived'
  verified_method TEXT,
  verified_by UUID, -- staff user id
  verification_at TIMESTAMPTZ,
  created_at TIMESTAMPTZ DEFAULT now(),
  updated_at TIMESTAMPTZ DEFAULT now()
);

CREATE INDEX idx_patients_name_trgm ON patients USING gin ((first_name || ' '
|| last_name) gin_trgm_ops);
CREATE INDEX idx_patients_dob ON patients (dob);

-- users
CREATE TABLE users (
  user_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  patient_id UUID REFERENCES patients(patient_id),
  display_name TEXT,
  email TEXT,
```

```
  phone_normalized TEXT,
  is_locked BOOLEAN DEFAULT false,
  mfa_enabled BOOLEAN DEFAULT false,
  created_at TIMESTAMPTZ DEFAULT now(),
  last_login TIMESTAMPTZ
);

CREATE INDEX idx_users_email ON users (lower(email));
CREATE INDEX idx_users_phone ON users (phone_normalized);

-- credentials
CREATE TABLE credentials (
  credential_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id UUID REFERENCES users(user_id) ON DELETE CASCADE,
  credential_type TEXT NOT NULL, -- 'password','pin','webauthn'
  password_hash TEXT,
  password_salt TEXT,
  pin_hash TEXT,
  created_at TIMESTAMPTZ DEFAULT now(),
  last_used_at TIMESTAMPTZ
);

-- external_identities
CREATE TABLE external_identities (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id UUID REFERENCES users(user_id) ON DELETE CASCADE,
  provider TEXT NOT NULL,
  provider_sub TEXT NOT NULL,
  created_at TIMESTAMPTZ DEFAULT now(),
  UNIQUE(provider, provider_sub)
);

-- proxy_access (caregiver delegations)
CREATE TABLE proxy_access (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  patient_id UUID REFERENCES patients(patient_id) ON DELETE CASCADE,
  delegate_user_id UUID REFERENCES users(user_id) ON DELETE CASCADE,
  granted_by_user_id UUID REFERENCES users(user_id),
  scopes JSONB,
  status TEXT DEFAULT 'active',
  start_at TIMESTAMPTZ DEFAULT now(),
  end_at TIMESTAMPTZ,
  created_at TIMESTAMPTZ DEFAULT now()
);

-- verification_evidence
CREATE TABLE verification_evidence (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
```

```
  patient_id UUID REFERENCES patients(patient_id) ON DELETE CASCADE,
  evidence_type TEXT,
  file_ref TEXT, -- link to encrypted object store
  hashed_name TEXT,
  uploaded_by UUID,
  uploaded_at TIMESTAMPTZ DEFAULT now(),
  retention_policy TEXT
);

-- staff_invites
CREATE TABLE staff_invites (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  patient_id UUID REFERENCES patients(patient_id) ON DELETE CASCADE,
  invited_by_staff_id UUID,
  invite_token_hash TEXT,
  delivery_channel TEXT,
  expires_at TIMESTAMPTZ,
  used_at TIMESTAMPTZ,
  created_at TIMESTAMPTZ DEFAULT now()
);

-- empi_records (store last known empi metadata for audit)
CREATE TABLE empi_records (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  patient_id UUID REFERENCES patients(patient_id),
  empi_payload JSONB,
  score NUMERIC,
  created_at TIMESTAMPTZ DEFAULT now()
);

-- merge_tickets
CREATE TABLE merge_tickets (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  source_patient_id UUID,
  target_patient_id UUID,
  status TEXT DEFAULT 'open',
  created_by UUID,
  approved_by UUID,
  created_at TIMESTAMPTZ DEFAULT now(),
  resolved_at TIMESTAMPTZ
);

-- attempt_counters
CREATE TABLE attempt_counters (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  context_type TEXT,
  context_value TEXT,
  count INTEGER DEFAULT 0,
```

```
  last_attempt TIMESTAMPTZ
);

-- audit_log (append-only)
CREATE TABLE audit_log (
  id BIGSERIAL PRIMARY KEY,
  actor_type TEXT,
  actor_id UUID,
  action TEXT,
  resource_type TEXT,
  resource_id UUID,
  meta JSONB,
  created_at TIMESTAMPTZ DEFAULT now()
);

-- index for audit query
CREATE INDEX idx_audit_actor ON audit_log (actor_type, actor_id);
```

Notes: - Sensitive fields (SSN, government ids) should be stored in `identifiers` or `verification_evidence` and encrypted at the application layer or via field-level encryption. - Use KMS/HSM to store encryption keys (Azure Key Vault / AWS KMS depending on infra).

---

4. Microsoft Entra External ID: tenant & app registration checklist (Phase 1)

Goal: Minimal Entra configuration so Entra is the auth mediator while Postgres remains authoritative.

Tenant Prep: - Create an External ID tenant or enable External Ids in existing Entra tenant for consumer flows. - Configure branding and company display name.

App Registrations: 1. **SPA (Patient Portal)** - Platform: Single-page application - Redirect URIs: `https://<app>/auth/callback` - Implicit flow: OFF; use Authorization Code + PKCE - Add `logout` redirect URIs 2. **Auth Service (Server-side)** - Platform: Web/API - Redirect URIs: for backend flows if needed - Expose an API scope `api://<id>/auth` for internal services 3. **Admin Portal (Hospital staff)** - Web application with elevated RBAC roles

Custom Attributes (User claims) - `extension_upi` (string) - `extension_patient_id` (GUID) - `extension_verified_method` (string) - `extension_roles` (array)

Identity Providers (Phase 1) - Google, Apple (optional), Microsoft (optional)

Custom Policies (stub) - Create custom policy templates for: UPI-login, OTP-login, Step-up orchestration. Initially these will call Auth Service endpoints (validate-upi, validate-stepup) via REST technical profiles.

Conditional Access & MFA - Configure baseline policies to require MFA for staff and for elevated scopes. - For patient flows, allow step-up to require MFA based on risk.

Audit / Diagnostic - Enable sign-in logs and forward logs to SIEM/Azure Monitor.

---

5. Auth Service (API surface — Phase 1)

Auth Service is a .NET Core microservice that validates Entra callbacks, implements UPI/password validation, orchestrates EMPI calls, and issues internal session tokens if desired.

Core Endpoints (Phase 1) - `POST /auth/validate-upi` — Validate UPI + password; returns user_id, patient_id, roles, and claim set. - Input: `{ upi, password, client_context }` - Output: `{ ok: true, user_id, patient_id, claims }` or `{ ok:false, reason }`

- `POST /auth/exchange` — Exchange Entra id_token for internal session token (optional)
- Input: `{ id_token }`
- Behavior: validate id_token signature, ensure provider_sub mapping in `external_identities`, upsert `users` row if necessary, return internal JWT.
- `POST /empi/match` — Call EMPI engine/service with demographics/contact and return candidate list.
- Input: demographic payload or contact
- Output: array of candidates: `{patient_id, upi, masked_name, score, reasons}`
- `POST /auth/stepup/candidates` — Convert EMPI candidate list to masked UI payload.
- `POST /auth/stepup/verify` — Verify Level 2 step-up (DOB + PIN + UPI_last4).
- Input: `{ candidate_upi, dob, pin, upi_last4 }`
- Output: `{ verified: true, user_id, patient_id }` or `{ verified:false, reason }`
- `POST /staff/create_patient` — Staff create patient (for hospital intake)
- Input: full demographics, staff_id, optional MRN
- Output: `{ patient_id, upi, invite_token }`
- `POST /staff/send_invite` — Send secure invite link to patient contact (enqueues SMS/email)

Auth Service responsibilities: - Validate Entra tokens and claims via JWT validation libraries. - Hash and verify passwords/PINs using Argon2id + per-user salt + pepper stored in KMS. - Maintain

`external_identities` mapping for provider_sub. - Enforce rate-limits on endpoints like `validate-upi` and `stepup/verify`. - Write audit events for all key identity operations.

---

6. Security & Compliance (Phase 1 must-haves)

- **Passwords/PINs:** Argon2id hashing (configurable cost), per-user salt, pepper in KMS.
- **Encryption:** TLS 1.2+/1.3 for all transit; DB at-rest encryption via disk-level + application-level encryption for PII.
- **Audit:** `audit_log` append-only; ship to SIEM with RBAC.
- **MFA:** Enforce for staff; patient MFA optional but recommended for high-sensitivity scopes.
- **Rate-limiting & bot protection:** reCAPTCHA for sign-up and rate-limits for OTP, step-up attempts.
- **BAA / vendor agreements:** Ensure signed BAAs with Microsoft (Entra) and Twilio (if used).

---

7. Phase 1 Implementation Tasks & Owners (sprint-ready)

Estimate: 2 full-stack engineers + 1 devops/security, 2 weeks.

Task list (detailed)

1. **Schema & Migrations** — create migrations for all Phase 1 tables and run in dev/test. (Owner: Backend)
2. **Extensions & Indexes** — enable `pgcrypto`, `pg_trgm`; create indexes. (Owner: DBA)
3. **Auth Service bootstrap** — create .NET project, add JWT validation, endpoints stubs for `/auth/exchange`, `/auth/validate-upi`, `/empi/match`. (Owner: Backend)
4. **Entra tenant setup** — create tenant/app registrations, custom attributes, baseline policies. (Owner: DevOps / Identity)
5. **External IdP config** — add Google sign-in test. (Owner: DevOps)
6. **Twilio / SMS provider stub** — integration or stub for OTP send/verify. (Owner: Backend)
7. **Seed data** — create script to insert test patients (including twin-like records). (Owner: Backend)
8. **Logging & Audit wiring** — ensure audit_log writes from Auth Service. (Owner: Backend)
9. **Security review** — initial threat model and controls checklist. (Owner: Security)
10. **Integration test** — end-to-end basic login flow using Entra + Auth Service. (Owner: QA)

---

8. Acceptance Criteria (Phase 1)

- DB schema applied in dev and all migrations succeed.
- Entra app registrations created and SPA OIDC + PKCE flow works against dev tenant.
- `POST /auth/exchange` validates Entra id_token and upserts `external_identities` to users table.
- `POST /auth/validate-upi` validates local credential (password) against hashed password in `credentials` and respects lockout.
- EMPI stub returns candidates for seeded twin-like patients.

- Audit logs created for signup, validate-upi, and staff create patient events.
- Rate-limiting and reCAPTCHA integration present for signup endpoints.

---

9. Deliverables & Next Steps

Deliver Phase 1 artifacts by end of Sprint 1: - SQL migration scripts (Phase 1) - .NET Core Auth Service skeleton with unit tests for token validation - Entra tenant configuration checklist and policy stubs - E2E test plan for basic sign-up / sign-in

After Phase 1 is validated in dev, we proceed to Phase 2 (EMPI engine, duplicate management rules, and UI for candidate selection).

---

10. Appendix — Useful implementation notes

- Use `pyproject` / `dotnet` build pipelines to automate DB migrations with CI/CD.
- For small-scale EMPI testing, create a microservice that implements blocking + Levenshtein/phonetic comparisons and returns synthetic scores.
- Keep `external_identities` authoritative for mapping Entra identities — do not store sensitive PII in Entra claims beyond `upi` and `patient_id` keys.

End of Phase 1 document.